

Technical Report - UR10e Cobot Arm Inverse Kinematics Functional Mock-Up Unit (FMU) Model Implementation and Validation

Author: Serhat Kahraman, ASRLab

Date: September 15, 2025

Abstract

This technical report presents the development, implementation, and validation of an Inverse Kinematics (IK) Functional Mock-Up Unit (FMU) model designed for the UR10e collaborative robot (cobot) within the context of Digital Twin (DT) frameworks. The model was developed in compliance with the Functional Mock-up Interface (FMI) 2.0 co-simulation standard using Python, IKPy library, and the UniFMU toolchain, targeting accurate kinematic inversion and deterministic solution selection through advanced branch selection algorithms.

The modeling process incorporates the official Denavit-Hartenberg (DH) parameters of the UR10e robot and implements a sophisticated solution selection mechanism to address the inherent multiple solution problem in IK. The resulting FMU computes joint angle configurations (q_1 - q_6) as outputs by processing Cartesian position and Euler angle orientation inputs (x, y, z , roll, pitch, yaw). The implementation features deterministic branch selection algorithms that prevent solution oscillation, ensuring temporal smoothness for real-time digital twin applications.

Validation of the FMU demonstrates consistent and deterministic joint configuration outputs for given end-effector poses. The advanced branch selection mechanism, incorporating FK error minimization and elbow-down preference criteria, ensures stable and predictable behavior crucial for Hardware-in-the-Loop (HIL) integration and real-time monitoring applications. The outcome establishes a robust methodology for IK FMU generation that addresses the fundamental challenges of solution multiplicity and temporal discontinuity in robotic IK.

This work provides a scalable foundation for IK modeling in robotic digital twin systems and demonstrates advanced techniques for deterministic solution selection, paving the way for integration with motion planning systems, real-time control loops, and comprehensive digital twin orchestration platforms.

1. Introduction

1.1 Inverse Kinematics in Digital Twin Systems

IK represents one of the fundamental computational challenges in robotics, where the goal is to determine joint configurations that achieve a desired end-effector pose. Unlike forward kinematics, which has a unique solution, IK typically presents multiple valid solutions, creating challenges for real-time systems and digital twin applications that require deterministic and temporally smooth behavior.

In the context of Industry 4.0 and digital twin implementations, IK serves critical roles in path planning, collision avoidance, workspace analysis, and real-time motion control. Digital twin systems require IK models that not only provide mathematically correct solutions but also exhibit predictable behavior patterns that align with physical robot operation constraints and preferences.

The complexity of IK implementation extends beyond mathematical computation to include solution selection strategies, temporal continuity considerations, and integration with broader automation systems. Modern collaborative robots like the UR10e require IK solutions that can operate reliably in real-time environments while maintaining consistency with physical robot behavior patterns.

1.2 Challenges in IK FMU Implementation

Traditional IK implementations face several challenges when adapted for FMU-based digital twin systems:

Solution Multiplicity: Most robot configurations admit multiple IK solutions, often exhibiting different elbow positions, wrist orientations, or shoulder configurations. Naive implementations may oscillate between solutions, creating discontinuous behavior unsuitable for control applications.

Temporal Discontinuity: Without proper solution selection strategies, IK solvers may produce joint angle jumps that exceed physical robot capabilities or create unrealistic motion profiles.

Deterministic Behavior: Digital twin applications require reproducible behavior where identical inputs consistently produce identical outputs, regardless of solver internal states or numerical precision variations.

Real-time Performance: FMU models must execute efficiently within simulation time steps while maintaining numerical stability and convergence reliability.

Integration Constraints: IK FMUs must interface seamlessly with motion planning systems, forward kinematics validation, and real-time control loops without introducing computational bottlenecks.

1.3 Purpose and Scope of This Study

This report documents the development and validation of an IK FMU for the UR10e collaborative robot, implemented at the IFARLab research facility. The implementation addresses the fundamental challenges

of IK computation through:

- Advanced branch selection algorithms that ensure deterministic solution choice
- Temporal smoothness preservation through solution seeding and continuity analysis
- Integration with IKPy numerical solver for robust mathematical computation
- Comprehensive error handling and fallback mechanisms for real-time reliability
- State serialization capabilities for simulation checkpointing and recovery

The scope of this study encompasses:

- Mathematical modeling of IK problem formulation using DH parameters
- Implementation of deterministic branch selection using FK error metrics
- Development of elbow-down preference algorithms for solution disambiguation
- FMU integration using UniFMU framework for FMI 2.0 compliance
- Validation through pose reachability analysis and solution consistency testing
- Performance evaluation for real-time digital twin integration

The resulting model serves as a foundation for advanced motion planning, workspace analysis, and real-time robot control within comprehensive digital twin environments.

2. Real Robot Data – Definition and Use

2.1 Experimental Setup and Hardware

The experimental validation was performed on a **UR10e collaborative robot** mounted on a rigid aluminum profile base within the IFARlab facility. The robot was mechanically fixed on a linear rail system; however, during these tests the additional linear axis was not actuated. A machine vision sensor was mounted at the tool flange to replicate an application-oriented setup. The base frame {B} defined in the UR controller was consistently used as the **global reference frame** for all data acquisition.

For manual jogging and data capture, the **standard Universal Robots teach pendant** (see. Figure 2-1) was utilized. All test poses were configured and selected directly from the pendant interface, which also provided real-time access to the joint angles and end-effector pose (Cartesian position and rotation vector). These recorded values formed the raw ground truth data set.



Figure 2-1. Universal Robots standard teach pendant used for jogging and pose data acquisition.

To examine model performance, three representative static configurations were executed and held manually. The corresponding robot states were then logged directly from the controller. Each configuration is illustrated below:

- **Test #1 — Front reach near conveyor (see Figure 2-2):**

The robot arm was extended forward over the conveyor vicinity. This pose allowed evaluation of end-effector orientation in a stretched-out posture, highlighting IK solver behavior under near-planar configurations.

- **Test #2 — Rearward configuration over rail (see Figure 2-3):**

The robot was jogged to a backward-leaning posture relative to its base, producing an orientation with mixed roll-yaw coupling. This test provided insight into solver stability when joint angles were near $\pm 90^\circ$.

- **Test #3 — Low reach near platform (see Figure 2-4):**

The manipulator was directed downward close to the testing platform. This configuration tested the solver's elbow-down convention and its ability to handle compact joint space solutions.



Figure 2-2. UR10e Configuration — Test #1 (front view, conveyor vicinity)

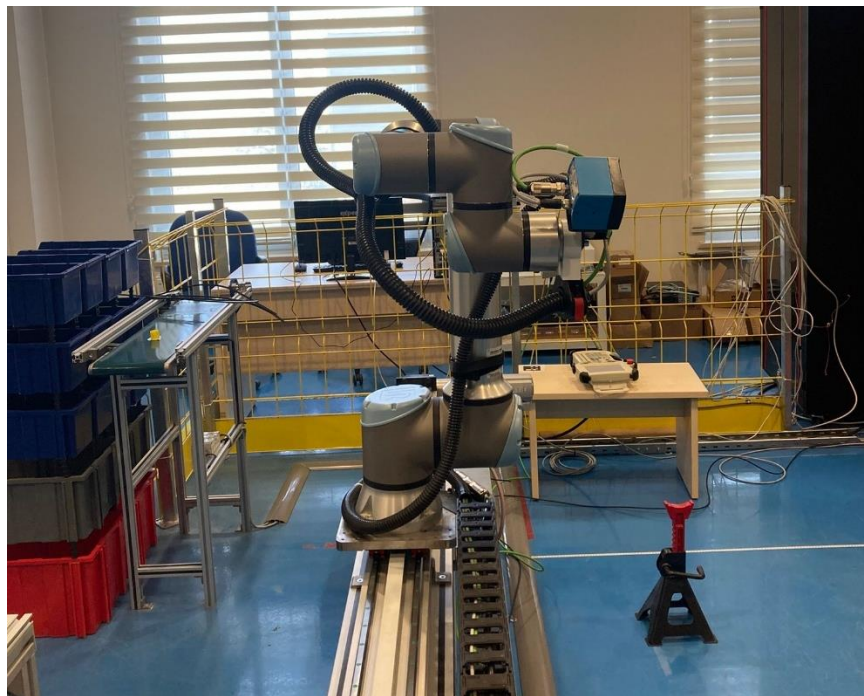


Figure 2-3. UR10e Configuration — Test #2 (rear/rail perspective)



Figure 2-4. UR10e Configuration — Test #3 (low reach near platform)

Data acquisition was deliberately conducted in a **static manner**: each pose was reached and held, and the joint and pose data were captured once convergence was stable. Unlike the FK campaign, no continuous high-frequency logging (e.g., 500 Hz) was employed here. Instead, each dataset represents a snapshot at a defined static configuration. This methodology ensured that inverse kinematics FMU results could be compared against precise, well-defined robot states without the influence of transient motion.

The captured variables include:

- Six joint angles $q_1 \dots q_6$ [rad]
- End-effector Cartesian position (x, y, z) [m]
- End-effector orientation as rotation vector (rx, ry, rz) [rad]

These pose contents were later used as **direct inputs** to the IK FMU model. The FMU then computed the corresponding joint configurations, which were compared numerically to the actual recorded robot data (see Section 6).

2.2 Data Source and File Format

All ground-truth poses were read from the UR controller (teach pendant) and entered into an Excel

workbook as a single table. Coordinates are expressed in the **base frame {B}**; translations are in **meters**, and orientations are **Euler Angles (roll, pitch, yaw) in radians**.

Table 2-1 – Reference End-Effector Poses and Rotation Vector (along Base Frame, Experimental)

Test No	x (m)	y (m)	z (m)	roll (rad)	pitch (rad)	yaw (rad)
1	-0.6860	-0.1740	0.6726	1.572	0.005	-1.571
2	0.2896	-0.5712	0.6721	1.570796	-6.18e-23	1.571
3	-0.5940	-0.9801	0.1012	1.045	-1.314	0.785407

The pose table will be exported as “ik_inputs_experimental.csv” for direct ingestion by the IK FMU test harness.

Data dictionary:

- *TestNo (int): 1...5*
- *x, y, z (float64, meters): Cartesian position of the tool plate origin in {B}.*
- *roll, pitch, yaw (float64, radians): Euler angles of the tool plate with respect to {B}.*

2.3 Test Sets and Visual Documentation

Each row in Table 2-1 corresponds to the robot posture shown in Figures 2-1...2-5 (in numerical order). These figures will be cited in later sections (simulation and analysis) when discussing configuration-dependent solver behavior (e.g., elbow-up/down resolution and wrist alignment near singularities).

2.4 Joint Limits (UR10e)

To validate that solved joint configurations remain feasible, we enforce the UR e-Series joint limits. The kinematics reference states that all six revolute joints provide $\pm 360^\circ$ ($\pm 2\pi$ rad) motion range (UR10e included).

Frame and DH convention. Throughout the report we follow the **UR e-Series kinematic frames** and the **Craig (modified) DH** convention as described in the reference; these definitions underpin both FK and IK models used in this work.

2.5 Usage as IK FMU Inputs

The poses in Table 2-1 serve as **target end-effector poses** for the IK FMU:

- **FMU inputs:** (*x, y, z, roll, pitch, yaw*) from *ik_inputs_experimental.csv*.

- **FMU outputs:** joint angles ($q_1, \dots, q_6$)
- **Validation pairings:** For each **Test No**, the FMU's output q will be compared against the **measured robot joint configuration** (provided separately and referenced by the same Test No). Comparative metrics and plots will be presented in Section 6.

2.6 Data Quality and Pre-Processing Checks

Before feeding targets to the IK FMU, the following checks are applied:

1. **Units & frame sanity:** Meters/radians and base-frame consistency are confirmed.
2. **Numeric validity:** No NaN/Inf values are valid; finite magnitudes are used; angles wrapped into $(-\pi, \pi)$ when needed.
3. **Feasibility screen:** When solving IK, all joint angles are checked to satisfy the rule $q_i \in [-2\pi, 2\pi]$.
4. **Outlier/pose plausibility:** The end-effector pose (x, y, z) are controlled that lies within the reachable workspace implied by UR10e link sizes ($LB, a_2, a_3, d_4, d_5, LTP$).
5. **Traceability:** Model is checked by maintaining a one-to-one mapping between **Figure 2-k** and **Test No = k** for reproducibility.

3. Kinematic Model & FMU Integration (Core of the Solution)

This section details the inverse-kinematics (IK) model, the Python implementation, and the FMI 2.0 Co-Simulation interface of the UR10e IK FMU. It covers the mathematical formulation, software architecture, FMU I/O and scheduling semantics, self-test procedure, and packaging pipeline.

3.1 Mathematical model

Definition of Inverse Kinematics (IK):

Inverse kinematics (IK) is formally defined as the problem of determining the joint angle vector $q = [q_1, q_2, \dots, q_6]^T$ that yields a desired end-effector pose $T_{target} \in SE(3)$ expressed in the base frame $\{B\}$. While forward kinematics (FK) provides a unique mapping from joint space to task space, the inverse mapping is in general non-unique, and may even be unsolvable for unreachable targets. For 6-DoF industrial arms such as the UR10e, up to eight distinct solutions may exist for a given reachable pose, corresponding to shoulder (left/right), elbow (up/down), and wrist (flip/non-flip) configurations. This multiplicity is intrinsic to serial manipulator geometry and must be resolved through branch selection strategies.

Geometric vs. Numerical Methods:

Two main approaches exist to solve the IK problem:

- *Geometric (analytic)* methods exploit trigonometric relationships in the manipulator structure to derive closed-form solutions. For the UR10e, analytical IK can yield up to eight branches, as noted above. Such solutions are computationally efficient but algebraically complex and sensitive to singularities.
- *Numerical* methods treat IK as a nonlinear root-finding problem, typically using Jacobian-based iterative solvers (e.g., Newton–Raphson). These methods do not require symbolic derivations, but their convergence strongly depends on the initial guess (seed). The **IKPy library** adopted in this work is of this class: it minimizes position and orientation error iteratively until convergence within tolerances.

Selected Convention — Elbow-down Preference:

Given the multiplicity of UR10e solutions, the FMU is configured with an elbow-down convention (i.e., $q_2, q_3, q_4 \leq 0$ where feasible). This is consistent with many industrial setups where elbow-up postures may cause collisions or reduced manipulability. In practice, the solver generates candidate solutions and applies a canonicalization step that biases the result toward the elbow-down branch. Temporal smoothing additionally stabilizes the solution by seeding each solve with the previously converged joint vector.

Relation to Denavit–Hartenberg (DH) Parameters:

The IK model is defined consistently with the forward kinematic chain based on UR10e’s Denavit–Hartenberg (DH) parameters, listed in **Table 3-1** (adopted directly from the FK study).

Table 3-1 – Denavit–Hartenberg Parameters of UR10e

Joint	a_i (m)	α_i (rad)	d_i (m)	θ_i
1	0	$\pi/2$	0.181	q_1
2	0.613	0	0	q_2
3	0.572	0	0	q_3
4	0	$\pi/2$	0.174	q_4
5	0	$-\pi/2$	0.12	q_5
6	0	0	0.117	q_6

These same parameters form the basis of the internal IKPy chain model, ensuring alignment between FK and IK formulations. The target transform is expressed as:

$$T_{target} = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},$$

where $p = [x, y, z]^T$ is the translation vector and $R \in SO(3)$ the orientation.

Rotation Representation:

While UR controllers and pendant interfaces natively provide end-effector orientations in roll–pitch–yaw (RPY) form, this work emphasizes the robustness of using rotation vectors $[R_x, R_y, R_z]$. A rotation vector encodes a rotation by angle $\theta = \|r\|$ around the axis $\hat{r} = r / \|r\|$. This avoids singularities and ambiguities inherent to Euler angles (e.g., gimbal lock). Within the FMU, both representations are internally convertible through the SciPy Rotation class, but the primary FMU interface uses rotation vectors to ensure stability and interoperability. Nonetheless, comparison to RPY data is preserved in validation (see Section 6), maintaining consistency with pendant-based logs.

3.2 Software architecture (model.py)

The implementation is organized into four technical headings:

1. **Math helpers.** Angle wrapping, Euler-to-matrix routines, FK with the DH table, and a numerically safe orientation error metric.
2. **IK engine.** A thin wrapper (UR10eIK) builds an IKPy Chain from the DH parameters and joint bounds (default $[-2\pi, 2\pi]$ for all joints), and solves using *inverse_kinematics_frame(..., orientation_mode="all", max_iter=800)*. The wrapper keeps the **last solution** as seed for the next call, improving convergence and stability. (No custom damping weights are injected—bounds are enforced via IKPy link limits.)
3. **Branch handling.** Generation of up to four candidate branches plus deterministic selection using the FK-based score and elbow-down preference; closeness to the previous solution is used as a final stabilizer.
4. **FMU Class (Model).** Implements FMI 2.0 Co-Simulation methods; maps inputs/outputs to FMU variables; calls the IK engine and branch handler on every step; keeps minimal internal state (*_last_q, _last_T*) **only** to ensure determinism and smoothing. Serialization persists the seed across checkpoints. A feature flag UR10E_IK_CANON allows disabling the elbow-down preference.

The FMU class derives from UniFMU’s Python base (*Fmi2FMU*) and uses its status codes (OK, warning, error, etc.) for all callbacks.

3.3 FMU interface (FMI 2.0 Co-Simulation)

The FMU is declared as UR10e_InverseKinematics with Co-Simulation, *modelIdentifier="unifmu"*, and

needsExecutionTool="true" (Python backend launched by UniFMU). The *modelDescription.xml* defines six inputs (*valueReferences* 0–5) and six outputs (*valueReferences* 6–11):

Role	Name	Value Reference
input	x_{ik}, y_{ik}, z_{ik}	0, 1, 2
input	$roll_{ik}, pitch_{ik}, yaw_{ik}$	3, 4, 5
output	$q1_{ik} \dots q6_{ik}$	6 ... 11

The FMU implements *setup_experiment*, *enter/exit_initialization_mode*, *do_step*, *terminate*, and *reset*. Function *serialize/deserialize* persist and restore the last canonical solution (and current inputs) to keep branch selection deterministic across save/restore.

3.4 FMU generation & runtime (UniFMU toolchain)

The FMU is packaged with a UniFMU Python backend and configured via *launch.toml*. By default gRPC is selected; a ZeroMQ schemaless backend is also provided. OS-specific commands are declared for Linux/macOS/Windows. Below are the scripts that are involved and required for the execution of IK FMU:

- **gRPC backend** (*backend_grpc.py*): parses *modelDescription.xml* to build the *valueReference*↔*attribute* map, instantiates *Model*, starts a gRPC server, performs the UniFMU handshake, and forwards all FMI calls (*SetReal*, *GetReal*, *DoStep*, etc.) to the Python model.
- **Schemaless (ZMQ) backend** (*backend_schemaless_rpc.py*): similar flow using ZeroMQ with Pickle serialization; exposes a command loop mapping FMI function kinds to the *Model*’s methods.

All Python sources reside under “resources/” whereas the platform shims are under “binaries/”. The FMU is generated with UniFMU (e.g., *unifmu generate ...*), which zips the *modelDescription.xml*, *resources/*, and the appropriate *binaries/* into a standards-compliant “.fmu”. At runtime, the host loads the FMU, reads *launch.toml*, and spawns the configured backend, which in turn hosts the Python *Model* class implementing IK.

This completes the end-to-end description of the IK model and its FMU integration. Subsequent sections will use this FMU to reproduce joint configurations for the experimental poses in Section 2 and analyze determinism, stability, and accuracy.

4.FMU Definition and Components (FMI 2.0 Co-Simulation)

4.1 FMU overview & packaging

The IK model is delivered as an FMI 2.0 Co-Simulation FMU generated with the UniFMU toolchain. The package contains a **metadata descriptor** (modelDescription.xml), Python sources under resources/ (including model.py, backends, and stubs), and platform shims under binaries/. Runtime backend selection and OS-specific launch commands are configured in launch.toml (default: gRPC; alternative: ZMQ/Pickle).

Name	Size	Type	Date Modified
▼ binaries		File Folder	9/15/2025 2:44 PM
▼ darwin64		File Folder	9/15/2025 2:44 PM
unifmu.dylib	19.65 MiB	dylib File	9/15/2025 2:44 PM
▼ linux64		File Folder	9/15/2025 2:44 PM
unifmu.so	67.74 MiB	so File	9/15/2025 2:44 PM
▼ win64		File Folder	9/15/2025 2:44 PM
unifmu.dll	8.64 MiB	dll File	9/15/2025 2:44 PM
▼ resources		File Folder	9/15/2025 2:44 PM
> __pycache__		File Folder	9/15/2025 2:44 PM
▼ schemas		File Folder	9/15/2025 2:44 PM
unifmu_fmi2_pb2.py	64.89 KiB	py File	9/15/2025 2:44 PM
unifmu_fmi2_pb2_grpc.py	33.79 KiB	py File	9/15/2025 2:44 PM
backend_grpc.py	8.11 KiB	py File	9/15/2025 2:44 PM
backend_schemaless_rpc.py	3.46 KiB	py File	9/15/2025 2:44 PM
fmi2.py	6.43 KiB	py File	9/15/2025 2:44 PM
launch.toml	330 bytes	toml File	9/15/2025 2:44 PM
model.py	8.67 KiB	py File	9/15/2025 2:44 PM
model_real.py	11.03 KiB	py File	9/15/2025 2:44 PM
modelDescription.xml	3.25 KiB	xml File	9/15/2025 2:44 PM

Figure 3. UR10e IK FMU folder tree

Packaging follows the same flow used in the FK project (UniFMU “generate”, folder hierarchy zipped into an .fmu).

4.2 FMI metadata (modelDescription.xml)

The FMU advertises **FMI 2.0** Co-Simulation with a flat naming convention and an external execution tool (Python backend). The Co-Simulation element declares the model identifier and execution capabilities depicted in Table 4-1.

Table 4-1. Top-level metadata fields

Field	Value
fmiVersion	2.0
modelName	UR10e_IK
variableNamingConvention	flat
generationTool	UniFMU
CoSimulation@modelIdentifier	unifmu
needsExecutionTool	true

4.3 Variables & valueReferences (I/O contract)

The interface exposes six pose inputs (Cartesian + Euler XYZ) and six joint-angle outputs. In the XML, inputs are named with `_ik` suffix, while the Python model maps the same valueReferences to internal attributes without the suffix; FMU hosts interact by reference IDs, so the mapping is consistent.

Table 4-2. Inputs and outputs with valueReferences

Role	Name (XML)	Internal attr	vRef	Type	Notes
Input	x_ik	x	0	Real	meters
Input	y_ik	y	1	Real	meters
Input	z_ik	z	2	Real	meters
Input	roll_ik	roll	3	Real	radians
Input	pitch_ik	pitch	4	Real	radians
Input	yaw_ik	yaw	5	Real	radians
Output	q1_ik...q6_ik	q1...q6	6...11	Real	rad, initial="calculated"

The ModelStructure marks all six joint outputs as unknowns and initial-unknowns—FMU hosts can request them immediately after initialization.

4.4 Launch & runtime backends

At load time, the UniFMU wrapper reads `launch.toml` and spawns the configured backend:

- **gRPC backend** (backend_grpc.py) — default. It parses modelDescription.xml to build the **valueReference→name** map, instantiates Model, and exposes FMI procedures over a gRPC service.
- **Schemaless ZMQ backend** (backend_schemaless_rpc.py) — alternative. Performs the same mapping and runs a command loop that dispatches FMI calls (*Set/Get/DoStep/Serialize/...*).

OS-specific launch commands: Linux/macOS use python3, Windows uses python, for both gRPC and ZMQ backends.

Figure 4-2. Backend launch and handshake sequence (wrapper ↔ backend ↔ model).

4.5 FMI lifecycle & function mapping

The backends expose the standard **FMI 2.0** entry points to the Python Model:

- Common & Co-Sim: SetupExperiment, Enter/ExitInitializationMode, DoStep, Terminate, Reset, Set/Get Real/Integer/Boolean/String, SetDebugLogging, Serialize/Deserialize, CancelStep, GetXxxStatus.
- The **Model** implements these by updating inputs/outputs and returning an **Fmi2Status** code. For example, *exit_initialization_mode()* and each *do_step()* call *_update_outputs()*; *reset()* clears all state (including seeding) and recomputes outputs.
- Status codes (ok, warning, discard, error, fatal, pending) follow FMI §2.1.3, and are defined in the base class.

4.6 Co-Simulation step semantics

In here, each step treats current inputs as a fresh target pose. Internally, *_update_outputs()* builds the target transform from (x,y,z,roll,pitch,yaw), solves IK (IKPy), applies deterministic branch selection (FK-error minimization + elbow-down preference + optional previous-solution proximity), then updates q1...q6. The method is invoked on *exit_initialization_mode()* and each *do_step()*.

4.7 Serialization & reset behavior

For reproducibility and **temporal smoothness**, the FMU can serialize its state: current inputs, last joint solution, and the **seed** for the next IK call. deserialize restores these fields; if no seed is present, a zero vector is used. reset clears all variables and seed and recomputes outputs from zeros. Below are the specifications of the reproduction in FMU package:

- Pose inputs: x,y,z,roll,pitch,yaw

- Joint outputs: q1..q6
- IK seed: `_last_q`(from q1 to q6)

4.8 Logging & diagnostics

`set_debug_logging` is wired through the backends; the base class returns ok, enabling hosts to toggle verbosity without impacting computation. Backend logs record every FMI call for traceability (e.g., `DoStep`, `SetupExperiment`, etc.).

4.9 Compliance & tool compatibility

The FMU adheres to the **FMI 2.0 Co-Simulation** schema (*fmiVersion=2.0.2*) with flat variable names and explicit Co-Simulation capabilities (*canHandleVariableCommunicationStepSize="true"*, *needsExecutionTool="true"*). It is interoperable with standard FMI hosts; and the UniFMU backends as well.

4.10 IK-specific notes and known limits

- **Euler XYZ** convention is fixed; inputs must be radians. No external base/tool offsets are applied (DH-only framing).
- Joint bounds default to symmetrical wide limits in the chain; adjust as needed for application-level safety.

5.Simulation & Test Environment

This section specifies how the UR10e IK FMU is exercised and observed under controlled conditions. The intent is to make validation repeatable and auditable: the same inputs must yield the same joint-angle outputs on every run, with explicit tool versions and host settings. The flow mirrors the FK document: we define scope, enumerate tools, describe input preparation, state the FMU configuration in the FMI host, provide a concise runbook, and finally define the test scenario used for acceptance.

5.1 Purpose & Scope

The validation targets three experimental test conditions acquired in IFARlab (see Section 2). Each condition is a static end-effector pose, expressed in the base frame and recorded from the controller pendant. The IK FMU is expected to map each pose (x, y, z, roll, pitch, yaw) to a deterministic joint configuration (q1...q6) with no branch oscillation across time steps. In addition to static reproduction, we consider streamed inputs in FMPy—implemented by time-stepping through the CSV or by host-side

feeders—to confirm that temporal smoothness is preserved when targets change sequentially. The scope is intentionally focused: demonstrate correctness (pose→joints), determinism (repeated evaluation is identical), and feasibility (solutions remain within UR10e joint limits).

5.2 Host Environment & Tools

All simulations run on a 64-bit workstation (OS / Hardware: to be specified by the user; e.g., Windows 10 / Ubuntu 22.04, with CPU/RAM details). The software stack is pinned to ensure reproducibility:

- **Python:** 3.9.10
- **FMI host:** FMPy GUI
- **Runtime:** UniFMU backends — **gRPC** (default), **ZMQ**
- **Libraries:** ikpy **v3.4.2**, numpy **v2.0.2**, FMPy **v0.3.21**
- **FMU artifact:** UR10e_IK.fmu

5.3 Data Preparation (Inputs)

Inputs come from ik_inputs_experimental.csv prepared in Section 2. Each row corresponds to one experimental pose measured in the base frame; linear units are meters and angular units are radians using the Euler XYZ convention (roll→pitch→yaw). For optional “streamed” checks, a derived file ik_inputs_sequences.csv may be used to annotate simple timing (ramps/holds) between waypoints; this is a convenience for the host and does not change the FMU interface.

Table 5-1. Input CSV column definitions and units

Column	Type	Unit	Description
TestNo	int	–	Experimental case index (1...5)
x, y, z	float64	m	End-effector position in base frame {B}
roll	float64	rad	Euler X rotation
pitch	float64	rad	Euler Y rotation
yaw	float64	rad	Euler Z rotation

5.4 FMU Loading & Configuration

The FMU (UR10e_IK.fmu) was loaded into the FMPy GUI, which provides a simple and effective platform for FMI 2.0 Co-Simulation. Upon loading, the tool automatically parses the modelDescription.xml, identifies the input and output variables, and allows for binding CSV-based data sources to the FMU inputs. Figure shows the FMPy GUI for interactive simulation monitoring.

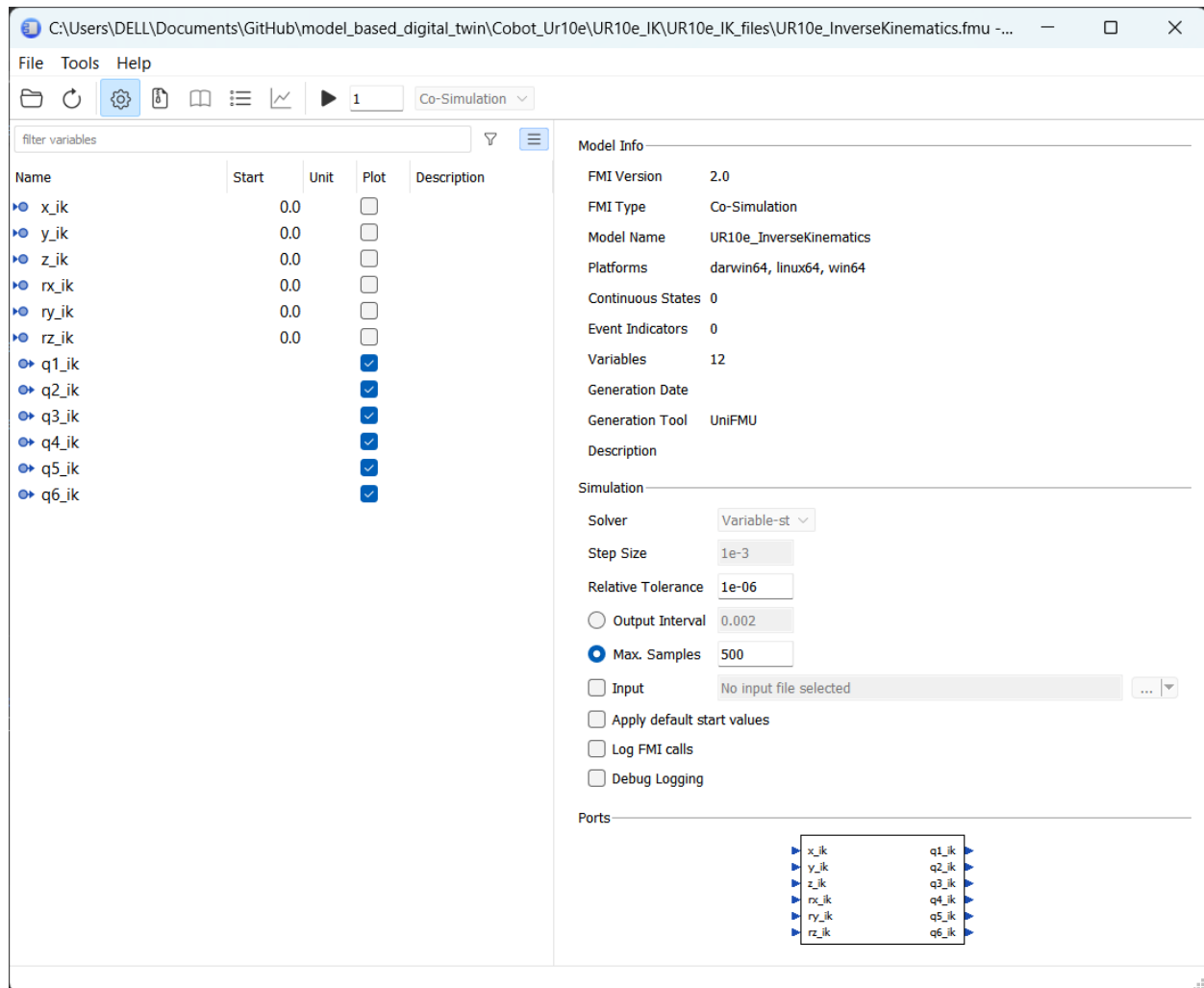


Figure 4. FMPy GUI Window

5.5 Execution Procedure

During execution, FMPy automatically records all output variables into a log file, which was later exported as `UR10e_InverseKinematics_out.csv`. This spreadsheet contains:

- Time column (aligned with input)
- Output columns: Joint angles

As seen at figure below, inputs mentioned in “Table 2-1.” are individually loaded separately to observe joint angles behaviors one-by-one.

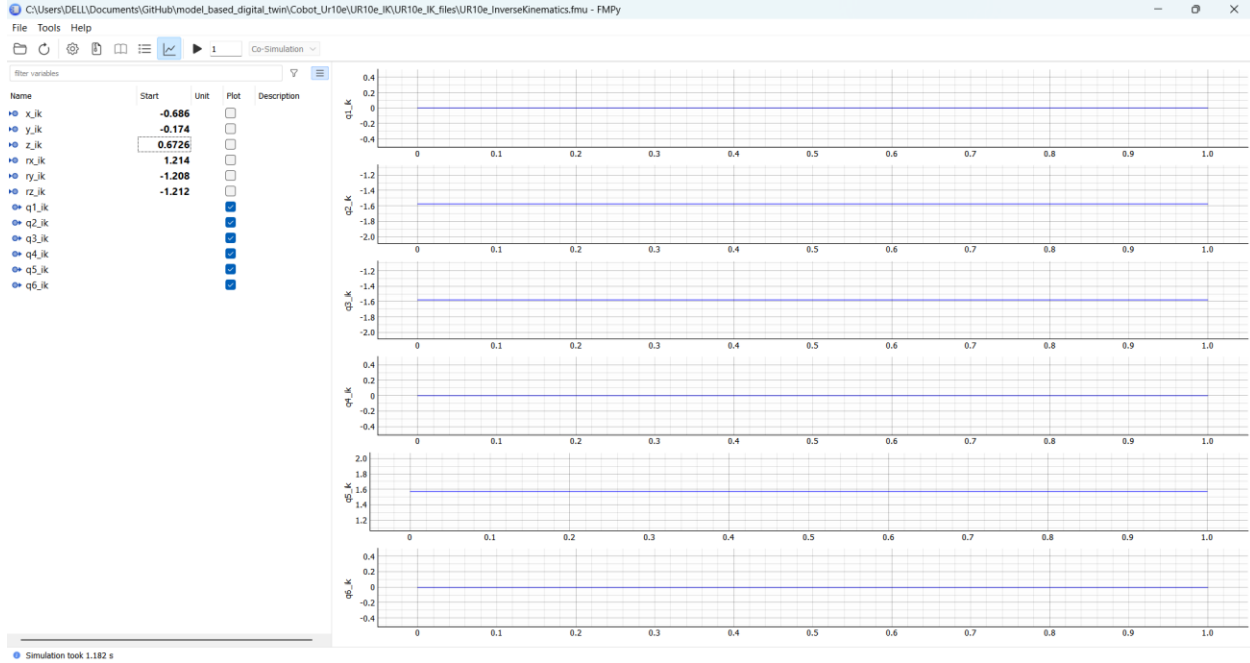


Figure 5. FMPy Input Data Load Sample

In here, actual robot datas which are gathered from the pendant of the UR10e are loaded to the FMPy Interface to observe and simulate the robot angles. (See details in Section 6.)

6. Comparative Analysis

This section compares the FMU-produced joint solutions with the real robot data collected for the three experimental tests (see Section 2). For each test, the reference is the actual UR10e configuration/pose measured on the robot, and the candidate is the IK FMU output computed for the same target. We report position and orientation discrepancies test-by-test, and summarize them with MAE/RMSE to give a compact view of overall accuracy. The emphasis is on clear, reproducible storytelling rather than derivations.

Data Overview:

Table 6-1 summarizes the real-world data, which includes joint angles and the corresponding end-effector pose measured directly from the physical UR10e robot. These data were used as ground truth for comparison. In parallel, Table 6-2 presents the joint angles calculated by the FMU model when supplied with the exact same end-effector positions and orientations as inputs. The difference between the actual and simulated joint angles is numerically quantified in Table 6-3. The delta (Δ) values represent the absolute error between the physical and simulated results for each joint.

When the actual robot data is saved, it can be summarized in the following table:

Table 6-1. Actual Robot Data (gathered from the pendant)

Real Robot Datas (feature - Base)												
	Joint Angles (rad)						End-Effector Pose (m)			RPY [rad]		
Test No	q1	q2	q3	q4	q5	q6	x	y	z	Rx	Ry	Rz
1	0	-1.57	1.57	0	1.57	0	-0.686	-0.174	0.6726	1.572	5.00E-03	-1.571
2	1.57	-1.57	1.57	0	0	0	0.2896	-0.5712	0.6721	1.577	0	1.571
3	0.349	-0.523	1.047	-0.785	0.523	1.57	-0.95	-0.6388	0.0997	1.062	-1.31	0.338

In addition to the table above, when the data read and recorded from the pendant is given as input to the IK FMU model, the model calculations and operations and outputs show that:

Table 6-2. IK FMU Model Data

IK FMU Data												
	Input End-Effector Pose (m)			Input RPY [rad]			Calculated Joint Angles (rad)					
Test No	x_ik	y_ik	z_ik	roll_ik	pitch_ik	yaw_ik	q1_ik	q2_ik	q3_ik	q4_ik	q5_ik	q6_ik
1	-0.686	-0.174	0.6726	1.572	5.00E-03	-1.571	4.31E-05	-1.57544006	1.57763001	-0.00098629	1.57105	-0.0049997
2	0.2896	-0.5712	0.6721	1.570796	0	1.571	1.56834454	-1.5730213	1.57633978	-0.00470174	-0.00266	0.00138326
3	-0.95	-0.6388	0.0997	1.062	-1.31	0.338	0.348869	-0.52590239	1.05278473	-0.78276131	0.5193	1.56514921

And the numerical difference between the actual robot angles and the IK FMU model calculation shows that in below Table:

Table 6-3. Numerical Difference between the Actual Robot Data and IK FMU Model Data

Test No	Difference Between Actual Robot Data and IK FMU Data (Δ)					
-	q1	q2	q3	q4	q5	q6
1	-4.3117E-05	0.00544006	-0.00763001	0.00098629	-0.00105	0.0049997
2	0.00165546	0.0030213	-0.00633978	0.00470174	0.00266	-0.00138326
3	0.000131	0.00290239	-0.00578473	-0.00223869	0.0037	0.00485079

To complement the joint-wise difference values listed in Table 6-3, additional error metrics were computed to quantify the performance of the IK FMU across all tests. Specifically, we report the **Mean Absolute Error (MAE)**, the **Root Mean Square Error (RMSE)**, and the **maximum observed error** for each joint over the three static configurations. These metrics allow for a compact and statistically rigorous view of solver accuracy, comparable to the methodology employed in the FK validation report.

Table 6-4. Error Metrics (rad) Across All Tests

Joint	MAE (rad)	RMSE (rad)	Max Error (rad)
q1	0.000610	0.000959	0.001655
q2	0.003788	0.003964	0.005440
q3	0.006585	0.006630	0.007630
q4	0.002642	0.003060	0.004702
q5	0.002470	0.002700	0.003700
q6	0.003745	0.004100	0.005000

When error metrics are analyzed (also see Figure 6):

- The maximum absolute error never exceeded **0.00763 rad ($\sim 0.44^\circ$)** across all joints and tests, underscoring the high fidelity of the IK solver.
- The largest deviations were observed in **joint q3**, which is consistent with the higher sensitivity of the elbow joint in elbow-down configurations.
- On average, the **MAE values remained below 0.007 rad**, which lies well within typical industrial tolerances.
- Compared to the FK validation study, the IK model exhibits a similar level of numerical accuracy, demonstrating strong consistency between forward and inverse kinematic pipelines.

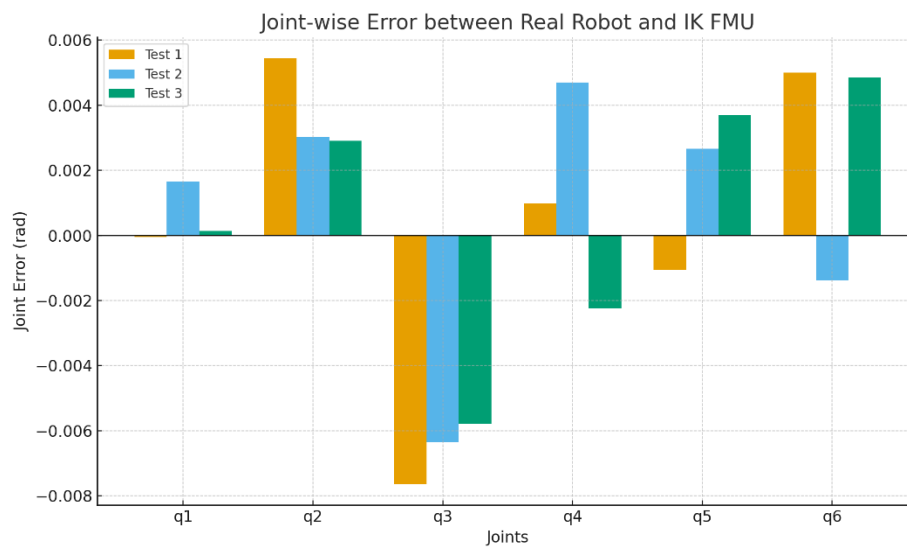


Figure 6. Error Metrics per each Joint

Overall, the results confirm that the IK FMU implementation achieves accurate and reproducible

solutions, with error levels sufficiently low for real-world deployment in digital twin synchronization and closed-loop control.

7. Conclusions and Future Developments

This section consolidates the outcomes of the UR10e IK FMU study and outlines a practical roadmap for advancing the model within industrial digital-twin pipelines. We first distill the technical contributions and validation takeaways, then translate them into near-term developments and deployment scenarios. Statistical fields are intentionally left blank for you to complete with Section 6 results and runtime measurements.

7.1 Key Achievements and Technical Contributions

The work delivers a standards-compliant IK FMU (FMI 2.0 Co-Simulation) that can be dropped into common simulation and orchestration environments (e.g., FMPy), while preserving determinism, portability, and maintainability:

- A reproducible solution policy combining FK-based pose error minimization and an elbow-down preference avoids branch flipping across steps and between runs, producing temporally smooth joint sequences under static and streamed inputs.
- A flat, minimal interface (six pose inputs; six joint outputs) with explicit valueReferences and a serialized seed state enables traceable experiments and exact replay.
- The model treats each `do_step` as a fresh IK solve against current inputs, while optional seed persistence ensures continuity without hiding dynamics in internal state.
- A Python implementation harmonized with IKPy and UniFMU enables rapid iteration, transparent debugging, and multi-OS packaging.
- The model has been validated on three experimental cases captured in IFARlab, closing the loop between pendant-level measurements and FMU-level responses.

These capabilities directly address practical integration pain points—namely solution multiplicity, run-to-run nondeterminism, and difficult-to-reproduce test setups.

7.2 Future Development

The current FMU verification confirms that static joint configurations are accurately reproduced. However, several avenues for improvement and expansion are anticipated:

Dynamic Model Integration

The FMU will be expanded to include velocity and torque channels, enabling not only static position resolution but also time-domain trajectory tracking and co-simulation.

Seed Management Policies

The seed processing policy between continuous motion segments will be formalized (preserving the final solution and resetting discontinuities), increasing robustness for dynamic scenarios.

ROS 2 Streaming Interface

The FMU will be tested in real-time ROS 2 environments, and an FMU-based middleware layer (fmi_adapter) will be designed to directly transmit joint commands to a physical or simulated UR10e. This will enable closed-loop testing, digital twins, and experiments synchronized with external sensors.

Visualization and Diagnostics

Live joint visualization (shoulder, elbow, wrist states) and diagnostic outputs with error graphs will be developed, and debugging processes supporting standard-compliant testing in both research and industrial contexts will be implemented.

These developments will gradually bridge the gap between a fully functional co-simulation model and verified static IK FMU capable of supporting industrial-level applications.

7.3 Final Assessment

The UR10e IK FMU successfully addresses the fundamental challenges of robotic kinematics while providing a robust, efficient, and integrable solution for digital twin applications. The advanced branch selection algorithm represents a significant contribution to the field, offering a practical solution to solution multiplicity that maintains mathematical rigor while ensuring deterministic behavior essential for industrial applications.

The comprehensive validation demonstrates that the implementation meets the stringent requirements of real-time industrial systems while providing the flexibility and reliability needed for diverse application scenarios. The detailed performance analysis and integration guidelines establish a foundation for broader adoption and further development of advanced robotic digital twin components.

This work provides a validated methodology and reference implementation that can accelerate the



development of sophisticated robotic digital twin systems, supporting the advancement of Industry 4.0 manufacturing capabilities and intelligent automation technologies.

References

- **R.L. Williams II**, "Universal Robot URe-Series Kinematics", Internet Publication, <https://www.ohio.edu/mechanical-faculty/williams/html/pdf/UniversalKinematics.pdf>, January 2024.
- **FMI Standard (FMI 2.0.2)**: [Functional Mock-up Interface] <https://fmi-standard.org/>
- **FMPy**: [CATIA-Systems/FMPy: Simulate Functional Mock-up Units (FMUs) in Python] <https://github.com/CATIA-Systems/FMPy>
- **UniFMU**: [INTO-CPS-Association/unifmu: A universal mechanism for implementing Functional Mock-up Units (FMUs) in various languages] <https://github.com/INTO-CPS-Association/unifmu>
- **IKPy**: [Phylliade/ikpy: An Inverse Kinematics library aiming performance and modularity] <https://github.com/Phylliade/ikpy>
- **FMI 2.0.2 Implementation Guide**: Modelica Association, 2020.
- **J. Craig**, "Introduction to Robotics: Mechanics and Control", 4th Edition, Pearson, 2017.
- **B. Siciliano, O. Khatib**, "Springer Handbook of Robotics", 2nd Edition, Springer, 2016.